

Q1. Multi-Attribute Indexing for an E-Commerce Platform

Question

An e-commerce platform experiences slow product searches. The **PRODUCT** table has 2 million rows with queries filtering by **Category** and **Price range**. Propose a multi-attribute indexing strategy using **B+ Trees**.

Answer

1. Problem Analysis

The **PRODUCT** table contains **2 million records**, so scanning the complete table for every search is inefficient.

Typical queries may be:

```
SELECT *
```

```
FROM PRODUCT
```

```
WHERE Category = 'Electronics'
```

```
AND Price BETWEEN 10000 AND 50000;
```

Without an index, the DBMS may perform a full table scan.

Therefore, a **multi-attribute B+ Tree index** can be created on:

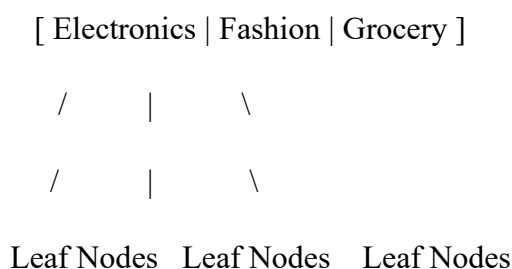
(Category, Price)

2. Proposed Index

```
CREATE INDEX idx_product_category_price
```

```
ON PRODUCT(Category, Price);
```

The composite B+ Tree index is arranged approximately as:



Category = Electronics
Price Price Price Price
5000 10000 25000 50000
Records

3. Why B+ Tree?

A B+ Tree is suitable because:

- It supports equality searches efficiently.
- It supports range searches efficiently.
- Leaf nodes are linked.
- It reduces the number of disk accesses.
- It works well for large tables.
- It can search Category first and then locate the required Price range.

4. Query Example

```
SELECT ProductID, ProductName, Price
FROM PRODUCT
WHERE Category = 'Electronics'
AND Price BETWEEN 10000 AND 50000;
```

The index first identifies:

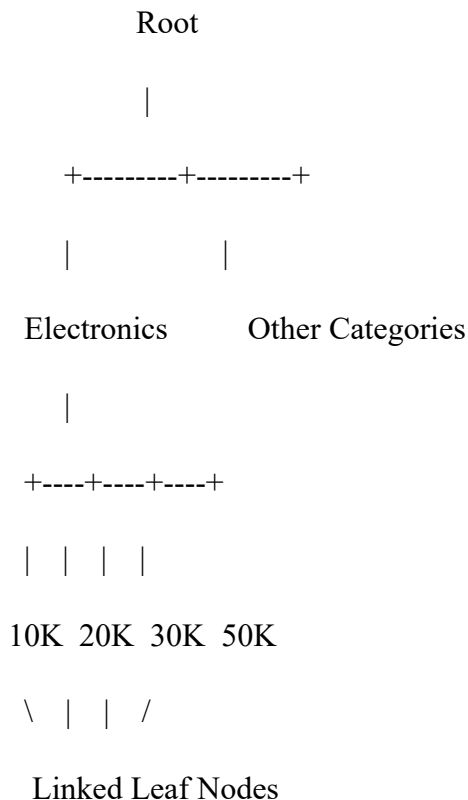
Category = Electronics

Then it searches the relevant price range:

10000 → 50000

Instead of checking all 2 million records, the DBMS can navigate through the B+ Tree and access only the relevant index entries and records.

5. Index Structure



The linked leaf nodes make range retrieval efficient.

6. Recommendation

The recommended index is:

```
CREATE INDEX idx_product_category_price
ON PRODUCT(Category, Price);
```

The column order is important. Since most queries filter by **Category first and Price second**, (Category, Price) is more appropriate than (Price, Category) for these queries.

Conclusion

A composite B+ Tree index on **(Category, Price)** is an effective solution for the e-commerce platform. It reduces search time, supports price-range queries efficiently, and avoids unnecessary scanning of the 2 million-row table.

Expected rubric performance: Excellent – accurate concept, industry application, SQL and diagram included.

Q2. Clustered Index vs Unclustered Index for Admission Date

Question

A healthcare system frequently queries patient records by date range. Analyse whether a clustered index on Admission Date or an unclustered index with range pointer would be more efficient.

Answer

1. Problem Analysis

A healthcare system may frequently execute queries such as:

```
SELECT *
```

```
FROM PATIENT
```

```
WHERE Admission Date BETWEEN '2026-01-01'
```

```
AND '2026-01-31';
```

This is a **range query** because it retrieves records between two dates.

Therefore, the organization of records on disk becomes important.

2. Clustered Index

A clustered index stores the table records physically in an order related to the indexed attribute.

For this problem:

Clustered Index on AdmissionDate

can organize records approximately as:

AdmissionDate

v	
+-----+	
Jan 01	
+-----+	
Jan 02	
+-----+	
Jan 03	
+-----+	
...	
+-----+	
Jan 31	
+-----+	

Records having nearby admission dates are stored close to each other.

Advantages

- Very efficient for range queries.
- Fewer disk I/O operations.
- Related records are physically close.
- Sequential reading of a date range is efficient.

3. Unclustered Index

An unclustered index stores index entries separately from the physical order of the table.

Example:

AdmissionDate	Record Pointer
---------------	----------------

|

+-----> Record 205

+-----> Record 890

+-----> Record 1450

+-----> Record 2001

The actual patient records may be located at different positions on disk.

For a large date range, many random disk accesses may be required.

4. Comparison

Feature	Clustered Index	Unclustered Index
Physical record order	Follows index order	Independent
Range queries	Very efficient	Less efficient
Sequential access	Excellent	Moderate
Disk I/O	Lower	Higher
Suitable for AdmissionDate range	Yes	Less suitable
Best use	Range-based queries	Selective lookups

5. Example

Suppose the query is:

```
SELECT *
```

```
FROM PATIENT
```

```
WHERE AdmissionDate BETWEEN '2026-01-01'
```

```
AND '2026-03-31';
```

With a clustered index:

Jan 01 → Jan 02 → Jan 03 → ... → Mar 31

The DBMS can locate the first date and sequentially read the required records.

With an unclustered index:

Index

|

+--> Disk Page 10

+--> Disk Page 75

+--> Disk Page 21

+--> Disk Page 150

+--> Disk Page 42

The DBMS may need many random accesses.

6. Recommendation

A **clustered B+ Tree index on AdmissionDate** is recommended.

Example:

```
CREATE CLUSTERED INDEX idx_admission_date
```

```
ON PATIENT(AdmissionDate);
```

Note: The exact SQL syntax for clustered indexes varies by DBMS.

Conclusion

For a healthcare system that frequently performs **date-range queries**, a **clustered index on AdmissionDate** is more efficient than an unclustered index because records with nearby dates are stored close together, reducing disk I/O and improving range retrieval.

Q3. Indexing Scheme for Social Media Posts

Question

A social media company stores user posts in a heap file. Posts are retrieved by PostID (unique) and by Hashtag (non-unique). Design an appropriate indexing scheme and justify your choice.

Answer

1. Problem Analysis

The system stores posts in a **heap file**, meaning records are not maintained in any particular order.

There are two important search requirements:

1. Search using PostID
2. Search using Hashtag

The characteristics are different:

PostID → Unique

Hashtag → Non-unique

Therefore, separate indexes are appropriate.

2. Index for PostID

Since PostID is unique, a **B+ Tree index with a unique constraint** can be used.

Example:

```
CREATE UNIQUE INDEX idx_post_id  
ON POSTS(PostID);
```

Structure:

```
PostID  
|  
[100 | 200]  
/  |  \  
|  |  |
```


50 150 250

| | |

Record Record Record

Each PostID points to exactly one post.

Benefits

- Fast exact search.
- Unique values.
- B+ Tree provides efficient lookup.
- Suitable for equality queries.

Example:

SELECT *

FROM POSTS

WHERE PostID = 1050;

3. Index for Hashtag

Hashtags are **non-unique**.

For example:

#AI → Post 101

#AI → Post 205

#AI → Post 309

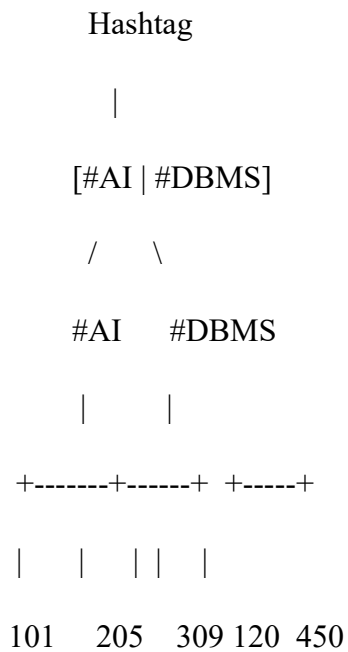
#DBMS → Post 120

#DBMS → Post 450

Therefore, a non-unique B+ Tree index can be created.

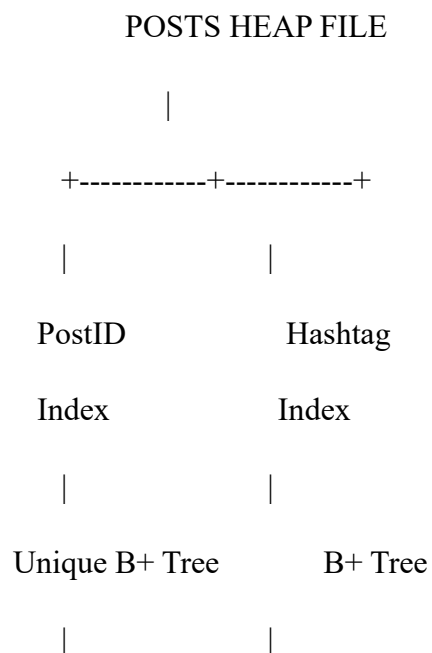
```
CREATE INDEX idx_hashtag
ON POSTS(Hashtag);
```

Structure:



The index points to multiple records for the same hashtag.

4. Complete Indexing Scheme



One record

Multiple records

5. SQL Implementation

```
CREATE UNIQUE INDEX idx_post_id  
ON POSTS(PostID);
```

```
CREATE INDEX idx_hashtag  
ON POSTS(Hashtag);
```

6. Search Examples

Search by PostID

```
SELECT *  
  
FROM POSTS  
  
WHERE PostID = 101;
```

The unique index directly identifies the required record.

Search by Hashtag

```
SELECT *  
  
FROM POSTS  
  
WHERE Hashtag = '#AI';
```

The non-unique index identifies all posts containing #AI.

7. Why B+ Trees?

B+ Trees are suitable because:

- They support fast exact searches.
- They support duplicate search keys.
- They are efficient for large datasets.

- Leaf nodes can be linked.
- They provide efficient ordered retrieval.

Recommendation

Use:

PostID → Unique B+ Tree Index

Hashtag → Non-unique B+ Tree Index

This is better than creating only one index because the two attributes have different search characteristics.

Conclusion

A **unique B+ Tree index on PostID** and a **non-unique B+ Tree index on Hashtag** provide efficient retrieval from the heap file. PostID searches return a single record, while Hashtag searches can efficiently locate multiple matching posts.

Q4. Static Hashing vs Dynamic Hashing

Question

A university enrollment system needs to handle 100,000 new records each semester. Evaluate static hashing vs dynamic hashing (extendible/linear) for this growing workload and recommend one.

Answer

1. Problem Analysis

The university enrollment system receives approximately:

100,000 new records / semester

Therefore, the database size continuously increases.

The hashing technique should:

- Handle growth efficiently.

- Minimize collisions.
- Reduce overflow.
- Provide fast insertion.
- Provide fast search.
- Avoid frequent complete reorganization.

2. Static Hashing

In static hashing, the number of buckets is fixed.

Example:

Hash Table

+-----+

| B0 |

+-----+

| B1 |

+-----+

| B2 |

+-----+

| B3 |

+-----+

| ... |

+-----+

| B9 |

+-----+

A common hash function is:

$h(\text{key}) = \text{key} \text{ MOD } \text{number_of_buckets}$

Problem with Growing Data

Initially:

10,000 records

may work well.

After several semesters:

100,000

200,000

300,000

...

The fixed number of buckets may become overloaded.

This causes:

More collisions

↓

More overflow chains

↓

More search operations

↓

Lower performance

To solve this, the table may need to be reorganized or resized.

3. Dynamic Hashing

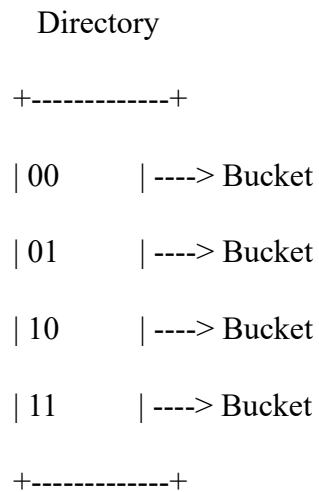
Dynamic hashing allows the number of buckets to grow as data increases.

Two common approaches are:

- Extendible hashing
- Linear hashing

4. Extendible Hashing

Extendible hashing uses a directory.



When a bucket becomes full:

1. The bucket is split.
2. The directory may be expanded.
3. Records are redistributed.
4. Other buckets do not necessarily need to be reorganized.

Advantages

- Dynamically grows.
- Good for rapidly increasing data.
- Reduces long overflow chains.
- Efficient search and insertion.

5. Linear Hashing

Linear hashing also grows gradually.

Initial:

B0 B1 B2 B3

As records increase:

B0 B1 B2 B3 B4



New bucket

Buckets are split progressively rather than rebuilding the complete table.

Advantages

- Gradual expansion.
- No large directory is required.
- Suitable for growing databases.
- Good insertion performance.

6. Comparison

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Grows dynamically
Growing data	Less suitable	Highly suitable
Overflow	Can increase	Lower
Reorganization	May be required	Usually gradual

Feature	Static Hashing	Dynamic Hashing
Search	Fast initially	Fast
Insertion	Good initially	Good
Scalability	Limited	High
Suitable for 100,000 new records/semester	No	Yes

7. Recommendation

For the university enrollment system, **dynamic hashing** is recommended.

Among dynamic methods, **extendible hashing** is a strong choice when predictable directory-based lookup and bucket splitting are desirable.

The system can grow like:

Semester 1

↓

Initial buckets

↓

Semester 2

↓

Bucket splitting

↓

Semester 3

↓

More bucket splitting

↓

Larger database

Instead of rebuilding the entire hash table, only necessary buckets are split.

8. Final Recommendation

100,000 new records/semester



Rapid data growth



Static hashing



Overflow increases



Performance decreases

Dynamic hashing



Bucket splitting



Gradual expansion



Better scalability

Therefore:

Recommended technique: Dynamic Hashing – preferably Extendible Hashing.

Conclusion

Static hashing is suitable when the database size is relatively stable. However, the university enrollment system has continuous and significant growth of **100,000 records every**

semester. Therefore, dynamic hashing provides better scalability, reduces overflow problems, and maintains efficient insertion and retrieval.